Computers and Mathematics with Applications 63 (2012) 525–535

Contents lists available at SciVerse ScienceDirect

Computers and Mathematics with Applications

journal homepage: www.elsevier.com/locate/camwa

A secure framework of the surveillance video network integrating heterogeneous video formats and protocols

Kyungroul Lee [a], Kangbin Yim [a,*], Mohammad A. Mikki [b]

[a] Department of Information Security Engineering, Soonchunhyang University, 646 Eupnae, Shinchang, Asan, 336-745, Republic of Korea

[b] Department of Electrical and Computer Engineering, Islamic University of Gaza, P.O. Box 108, Gaza, Gaza Strip, Palestinian Authority

ARTICLE INFO

Keywords:
Video surveillance
Heterogeneous video
Networked camera
Privacy masking
CCTV
CMS

ABSTRACT

Recently, IT technologies are becoming focused on different traditional industries. The large scaled intelligent video surveillance system is one of them and it integrates a large number of digitalized CCTV [1] devices through the Web. However, existing video devices have been deployed for many years by different vendors as different models with different specifications. To integrate these heterogeneous devices, the centralized management server (CMS) and its clients need a specialized architecture to deal with different types of media encodings and connection protocols etc. In particular, well-defined access control mechanism is required for a large scaled surveillance video network. In this paper, we designed and implemented a server that incorporates the architecture integrating and delivering multiple video streams from different types of video devices to multiple clients and securing the access to the video streams.

© 2011 Elsevier Ltd. All rights reserved.

1. Introduction

The video surveillance system was originally only a CCTV and has been regulated for providing a single channel closed circuit security monitoring service on public misdemeanors such as illegal car parking, illegal waste dumping etc. for where there are no security staff hired or for where collaborative work is required [1]. Hence, video surveillance systems have contributed to decreasing rates of illegal violations and crimes. However, the communication society has required more and more camera connections in many different fields including farmlands, factory assembly lines or disaster sites and the cameras have become common as an essential component of our society.

With recent advances in hardware and software, video surveillance cameras are digitalized to capture real-time images and have been improved to integrate into modern society for the centralized intelligent video surveillance system. In particular, the connections between the cameras and users are maintained through the web-based services on the Internet as a server and a client, respectively.

One of many disadvantages of traditional video cameras is that they are analog and they have much noise and distortion during transmission and manipulation. In contrast, modern web-based networked cameras are all digitalized and provide no quality degradation except from image compression. Furthermore, they don't need additional cables and equipment compared to the classical CCTV systems when the video information is distributed or aggregated, which provides a certain convenience when they are installed and maintained.

This convenience and social requirement have made digital surveillance cameras much more prevalent during the couple of decades. Because of these reasons, the centralized management system (CMS) was required as a specialized integrated client to cameras for collecting and managing the multiple video streams from multiple cameras.

* Corresponding author.
E-mail addresses: carpedm@sch.ac.kr (K. Lee), yim@sch.ac.kr (K. Yim), mmikki@iugaza.edu.ps (M.A. Mikki).

0898-1221/$ – see front matter © 2011 Elsevier Ltd. All rights reserved.
doi:10.1016/j.camwa.2011.08.048

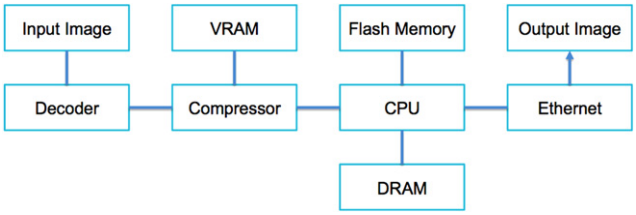

Fig. 1. Hardware structure of a networked camera.

Meanwhile, several severe problems have been found in developing and deploying the new centralized intelligent video surveillance system. Although researchers have contributed many efforts to the video surveillance systems with new protocols and compression algorithms to provide the best service connections during the development [2], these approaches were unfortunately considered a failure when they started to connect multiple cameras and multiple clients in an integrated network.

The existing networked camera's performance and bandwidth have proved that they are limited in the small number of client connections to a camera. Management inconveniences and security problems have arisen where the cameras have been installed at remote places. Because many networked cameras were connected through the ADSL with DHCP enabled, their IP addresses were unnoticeably changing and maintenance was difficult. In particular, because they are isolated from regular updates and a secure environment, they can become victims of attackers easily.

To resolve the above problems, a secure proxy server was introduced to be deployed within a protected network environment to hide cameras' connection information including IP addresses as well as IDs and passwords and to securely relay video information from cameras to clients. In this server type CMS configuration, a client connects to the designated server instead of a camera. In case of disaster monitoring or traffic broadcasting where multiple clients need to connect to one or multiple cameras for open video services, adding a distribution function can enhance the above proxy server [3].

Even though all of the above obstacles are resolved, there remains a most severe problem that makes it hard to share the open video images when multiple clients are connected to multiple cameras through a secure proxy server. Because of the incompatibility due to the variety of methods and types of video cameras from different vendors installed over decades, it is a difficult job to acquire, integrate, distribute and restore the heterogeneous video formats through the heterogeneous connection protocols from various cameras to various clients.

The CMS server is usually deployed along with a new installation of the same type of new multiple cameras for a new site. However, to integrate the existing multiple cameras or sites, a new adaptation mechanism is still required for different types of cameras from different vendors [4]. There we have experienced several frontier projects supported by the government that replaced hundreds of existing cameras each because of the incompatibility. It seems reasonable to replace some cameras other than one type of camera dominant in number with the same ones just for a small site. However, for a globalized intelligent video surveillance network, which is required these days, new software architecture is required to seamlessly integrate different types of cameras.

Even though all integration of the heterogeneous videos is completed, a reasonable access control mechanism is required to arbitrate the approaches from different types of users to different levels of sensitivity to the video information. The levels of sensitivity here may apply to the whole images from a specific camera or a part of an image from a camera when it has privacy masks. In a normal case, there will be at least three different levels of access privileges: general guests, administrators and prosecutors.

Therefore, this paper briefly surveys current situation of the video surveillance systems as well as their operating environments, suggests the architectural framework to integrate different types of cameras where the unified intelligent camera network is required and also proposes a security solution for the framework. The implemented prototype server has been deployed for the government to realize a broadcasting network for a transportation visualization system and a global disaster surveillance system.

2. Service structure of the networked camera

2.1. Basic structure of the surveillance camera network

A network camera enables us to provide monitoring of scenes by creating a digital formatted data for a series of images from the inputted analog video and to transfer these digital images to be seen by users. The internal hardware structure of a networked camera is shown in Fig. 1 [5]. Digital images need to be compressed before being stored in memory. These stored images are treated as a sequence of files and are provided to users through a network connection by the software running on the CPU.

Originally, networked cameras have been incorporating their own individual protocols and providing dedicated API libraries to give accessibility to users. These protocols were usually designed based on simple socket connections. For this

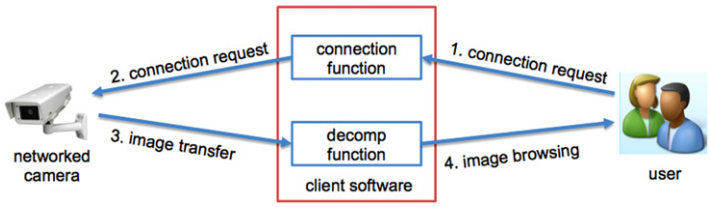

Fig. 2. Client operations to the networked camera.

Fig. 3. HTTP-based video service.

Table 1
Networked camera management information.

Field	Description
Connection IP	Networked camera's IP
Connection port	Networked camera's port
ID	User account
Password	Authentication password
Description	Location information installed

kind of camera, client software needs to implement all the sequences to request the images directly to the server. One sample sequence is illustrated in Fig. 2 and it may have the following steps.

Step 1. User requests to access the network camera.

Step 2. Client connects to the network camera through the connection function.

Step 3. Networked camera transacts images to the client, which were compressed by their own algorithm.

Step 4. Client services previously received compressed images to the user by the decoding function.

Socket-based individual protocols required too much overhead for client software developers. Subsequently, the architecture of the network camera has changed for easy connections. Therefore, comparatively new models have incorporated a HTTP based web server for connection and control and RTP facilities for video streaming [6]. In general, the HTTP is a set of rules for transferring files including text, images, sound, video, and other multimedia files. Because the networked camera deals with several different media, this transfer method has been applied to networked camera communication services. The Fig. 3 describes the process of transactions for the image service between user and camera based on the HTTP protocol.

The HTTP based protocol requires for client software developers only to organize a text based script to connect to the server and enables them to be developed easily. In particular, these types of cameras have incorporated a common gateway interface (CGI) for vendor-specific control functions and a designated function can be issued by sending a script with a parametric structure to the server through the HTTP protocol.

The client software for the networked camera has three different tasks: connection, decompression and management. The connection task is for keeping a connection to a camera server steady, requesting images to the server periodically and reordering the received images sequentially. The decompression task is for restoring the original images onto the user's window from the compressed images. The management task is for registering new cameras and users, manipulating databases for them and measuring the statistical parameters for maintenance. Even though they are functionally independent of each other, most manufacturers implemented these tasks in an integrated function module to combine them in one ActiveX because this makes it easy to release new software through the Internet.

The database used in the management function in particular has connection information that is gathered when a new camera is registered. The connection information includes the camera's IP address and connection port, user authentication parameters such as the user ID and the password as well as a description for the installed location of the camera. This information is defined in Table 1 in detail.

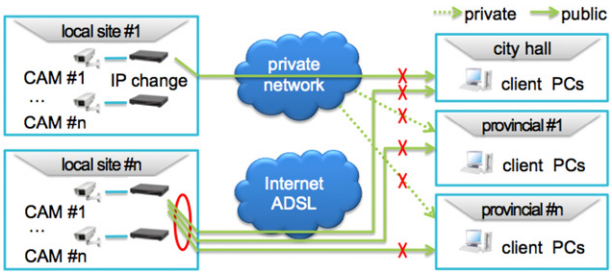

Fig. 4. Cases of access denials to networked cameras.

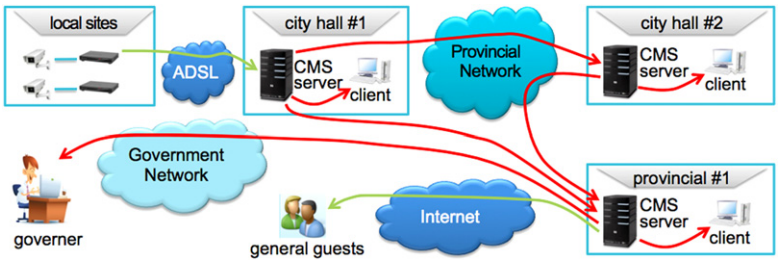

Fig. 5. Scenario for a large scaled public surveillance video network.

2.2. Current status of service architecture

The networked cameras are usually deployed at distant locations and connected to the Internet through a low bandwidth cable such as ADSL although a small amount of others are connected through the leased lines.

This service does not provide any security where this method of Internet accessibility to the image information could be shared with the public. However, the video network camera whose location is in a separate video network area could be installed with high security and privacy. To access a separate video network camera in another area, it is illegal because they are in separate configuration areas. Also, these concepts are difficult to integrate management in disaster response cases and it is difficult to implement them in regional cooperation. Fig. 4 describes the video shared information problem.

Many networked cameras are currently connected to the ADSL, which supports an upstream bandwidth of less than hundred kilobytes even though most camera traffic is upstream. Transmitted video image information can be shared by users within the entire network. The regional network of existing cameras that are shared among multiple users on the Internet generates a traffic load by the transmitting cameras due to video transmission requests. This traffic load might generate image quality degradation. Fig. 4 describes bandwidth increase in networked cameras due to the problem described above.

Normally, the network cameras are installed in distributed areas. If the IP address of the video camera changes, then each user needs to change his/her camera configuration in order to be able to continue camera accessibility. This change of camera IP address makes it difficult to notify multiple users of such a change. As a result, the integrated video surveillance system should display the IP addresses of cameras continuously to the administrator in order for the system to operate transparently to users. Fig. 4 also shows the problem of the changed CCTV camera IP address.

3. Framework for heterogeneous integrated camera network

3.1. Overall structural objective

For a large scaled public surveillance video network, several governmental entities are engaged from local sites to the government network. To design a whole framework for the required service, we need to find the possible traffic patterns. Fig. 5 shows the simplified entities and their traffic patterns. The connections are only for cameras on the local sites and for general guests on the Internet are public and the others are inside a secure network. Video information from the local sites is gathered to the CMS server in the city halls and then they are selectively collected at the provincial hall. Users with different privileges can access the video information from the CMS server at the provincial hall.

3.2. Server structure

Analyzing the basic architecture of the network cameras, their APIs, image compression algorithms and protocols may confuse in many ways. However, it is required to analyze, evaluate, classify the existing features and design a unique coherent architecture for a heterogeneous environment such as the one described above. To derive a reasonable server structure, analysis on current system needs to focus on the client organization and its programming environment.

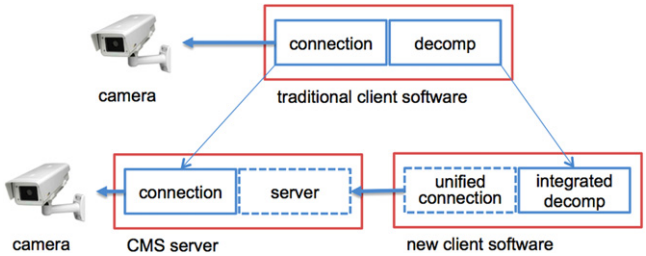

Fig. 6. Library needs to be separated for new framework.

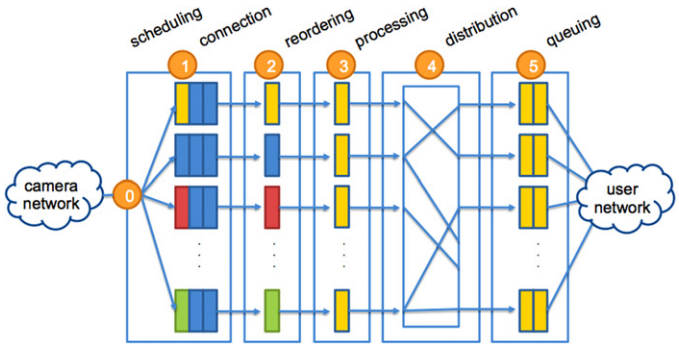

Fig. 7. Internal structure of the server processes.

Generally, camera manufacturers provide API libraries to encourage client software development. Some of the libraries have a part of portable modules for both connection and decoding, which are dynamically inserted into the client software. It makes the time-to-market very short because several lines of script can access the dedicated cameras if they have detailed specification for the software interfaces. It is also easy to construct a CMS and helpful when the CMS integrates multiple cameras even from different manufactures only when the server needs to be connected from just one client. In this case, the client script simply embeds portable modules that were already published on the server for each camera then the modules will be downloaded and executed on the client platform.

When multiple clients want to connect to the same camera, problems arise. As mentioned, effectively only one connection is allowed to a networked camera. To overcome this limitation, a server type CMS is required and it needs to gather the video stream from a camera and distribute it to multiple clients. In this situation, it is very difficult for the CMS to provide the same functionality as was provided on various cameras because heterogeneous connection protocols and media formats are transacted between the CMS server and clients in this approach. Therefore, an architectural framework should be designed to provide a flexible incorporation of various cameras for multiple users.

The traditional monolithic client libraries need to be redesigned for the new framework. It is essential for camera manufacturers to provide a separated set of connection and decoding modules for the CMS server and the clients, respectively. This eliminates emulating camera functionality on the CMS for clients in case the existing libraries are used for the CMS. The CMS server can only be responsible for connections to cameras using connection modules and bypasses all the media information from cameras to multiple clients. In this approach, the CMS server plays a role as a media switch. Even though the CMS server needs to brew connection modules if camera manufactures didn't provide them, a unified connection protocol needs to be designed between the CMS server and clients and an integrated portable module should be distributed to the client software, it would be simpler than developing a number of camera emulators. Fig. 6 shows the concept.

The concept explained above is helpful for providing interoperability of the networked camera. For interoperability, standardization is required in parallel to assuring manufacture's individual competing efforts to enhancing advanced private features. It means the standardization should be done with APIs, not with camera architecture itself.

On the CMS server side there are many considerations. Because of this reason, the server is designed to have six separated functional stages. Internal structures and information flows in the prototype implementation of the server are shown in Fig. 7. When multiple cameras are registered and managed, on-demand connection or pre-connection is applied. In the on-demand connection, the connection request to a camera is issued only when a client asked for the camera. Although this type of connection minimizes number of simultaneous connections, it takes delays the connection setup too much. Pre-connection sets up and maintains possible connections to cameras. Later connection requests from clients take less delay to be serviced. Only round robin scheduling is required for the demanded connections. Other considerable ones are to provide matching parameters for different connection protocols and manage correspondent buffers for different image formats. For some video streams from the cameras using UDP datagram, timestamp-based image reordering is required and this process needs several buffers and causes delays.

Table 2
Networked camera information.

Field	Type
Cam_no	Registration number
Cam_location	Installed location
Cam_ip	IP address
Cam_port	Port number
Cam_ch	Channel number
Cam_id	ID
Cam_pass	Password
Cam_model	Model name
Cam_group	Group
Cam_url	URL address
Cam_ptz	PTZ control information

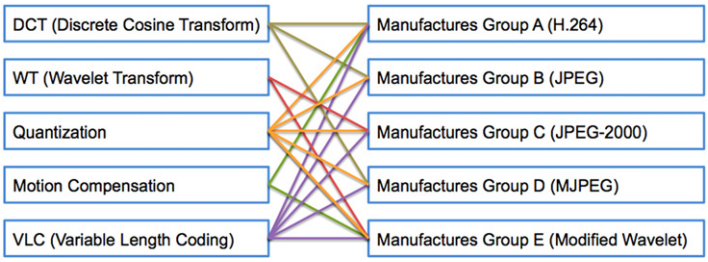

Fig. 8. Manufacturer-specific compression algorithms.

Queuing is required between the server and the client software for each connection to support analogous service. The number of buffers for queuing is determined by the policy of the jitter management. In case the multiple clients ask for the same video stream from a camera, the video information is copied to multiple queuing buffers on the distribution stage. In the processing stage, several selective functions are provided. These functions include frame rate scaling, privacy masking and encryption. Frame rate scaling is an alternative for the jitter management on the queuing stage. Privacy masking in this stage is post-compression privacy masking and it is sometimes very difficult and gives too much overhead because it needs to find adequate marker codes. Rectangular masks are only possible as well.

A unified connection protocol is also required for client software to connect to the CMS server. In particular for a networked camera to be connected with clients on a networked management system, it essentially needs the camera's IP address, port number and user ID and password to authenticate the permission to network camera's PTZ(Pan/Tilt/Zoom) [7] etc. It would be more convenient and recommended to reseal the authentication process for clients as groups with different permission privileges [8]. Table 2 shows a summary of the required fields for an abstracted connection.

Additional consideration is to support for cascading between CMS servers when they share the same video streams at different locations. To resolve this problem, we modeled the CMS server as a virtual networked camera based on the unified connection protocol and provided a connection function for the CMS server itself to connect to another CMS server. Any number of CMS servers can logically be cascaded through this idea even though the accumulated delay is getting severe if several servers are relayed.

3.3. Client structure

As mentioned above, client software is composed of two different main parts: a unified connection module and an integrated decompress module. In contrast to the connection module on the CMS server that is composed of the connection functions from the manufacturer's libraries, the connection module is for the unified connection protocol as a counterpart of the server's protocol. A more important one to focus on here is the integrated decompress module.

According to the analysis on more than ten camera manufacturers' compression algorithms, five major algorithms were dominant. Fig. 8 shows a summary of the image compression algorithms to design a decompression module for clients. Fig. 9 shows the client structure with unified connection protocol and dedicated decompression algorithms.

Several considerations are especially required on designing the decompression module for clients. For the networked camera based on the JPEG compressed images, the client could completely decompress an image independently. However, MPEG or modified Wavelet based images are transacted by components independently from the key frame (I-Frame) to delta frame (P-Frame) only if some changes occur in the information [5]. This means that if the client didn't receive any key frame from the camera, the client itself is unable to decompress the delta frame image [9,10]. Therefore, this integrated client needs to receive a key frame and keep it until the delta frame is resolved. The sequence for this process has the following flows.

Step 1. The server connection module performs the authentication procedure for the selected camera and periodically maintains the connection.

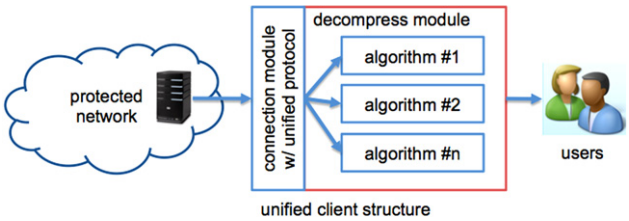

Fig. 9. Unified client structure.

Step 2. The images are received in a predefined format and a dedicated decoding module is selected as classified.
Step 3. Image buffer is allocated to the service.
Step 4. Decode the image and administrate the image sequence to prevent inversion based on the inserted time stamp in the images.

4. Framework for multi privileged access control

4.1. Basic requirements

Video surveillance systems are installed mainly to prevent criminal acts through providing a secure image service to the user to monitor protected places. This secure image service has been automatically enabled or is comparably easy for the traditional CCTV system because they are physically authorized for limited accessibility.

Now, networked cameras are physically connected to a public network and hence, an insistent access control policy especially for a large scaled video surveillance network should be considered, which is available only when the devices are all encryption and decryption aware. The video surveillance network includes various types of equipment devices that are different in performance and functions and it makes the public key mechanism is very difficult to be adopted. Therefore, we considered a private key mechanism based on Kerberos's protocol.

According to the key management technology, secure key handling is the core activity to be managed. In particular, secure private key distribution for participating entities is most significant. Furthermore, the key plays a very important role in securing images and needs to escape from exposure and eavesdrop by attackers. If the key is stolen by an attacker, then he could possibly sniff monitored images at all times. The attacker in this situation could resend a corrupted image to the client. In this case, the existing key should be discarded and a new key needs to be reissued.

There is an urgent need to research the field of key management for access control, which is currently not efficient for video surveillance systems. In this paper, we suggest a key management policy for a large scaled surveillance video network. The access control policy includes two different types of entities: the initiator and the target. The target can be a camera or the video information from the camera. The initiator can be one of the users accessing the targets.

The first one is to provide multiple levels of access privileges for different users. In many cases when the video surveillance is deployed, the privacy invasion problem arises. Therefore, the privacy masking facility is incorporated into some advanced recent models of networked cameras. No matter how the privacy masks are defined in these cameras however, these cameras currently mask out partial privacy-sensitive images but do not support privacy unmask or restore functions yet. However, encryption of an image can be expanded to privacy masking and restoration. Therefore, a crypto approach is implemented in this paper to be used for user authentication and permission as well as for whole image encryption and partial privacy masking.

4.2. Key management server

In this section, we describe the proposed key management server that is used for key issuance, distribution, maintenance and discarding. The surveillance video network ranges are very broad with regard to the number of users and cameras. This means it contains a lot of connections and information to deal with and various connection trials are requested. Due to this reason, a set of keys has to be managed separately from camera and user management. Therefore, the key management is maintained through a separate database that is different from the database used to store client and network camera information.

The key management server is engaged with all the management aspects of the keys among the network cameras, CMS servers and clients or other possible equipment devices. However, here we describe the key management among clients, the CMS server and the key management server because the existing cameras have no facility to deal with the key-related function as well as encryption. Fundamental flow is the same even when the participating entities are changed.

Fig. 10 shows key distribution process in the surveillance video network. Through the process, a shared key is generated at the key management server when a client requests a key and the key is distributed both to the client and the CMS server for encryption and decryption, respectively. Tables 3 and 4 show notations for Fig. 10.

The step-by-step sequence of the key distribution process is the following.

Step 1. Client inputs the ID (candidate) for user authentication.

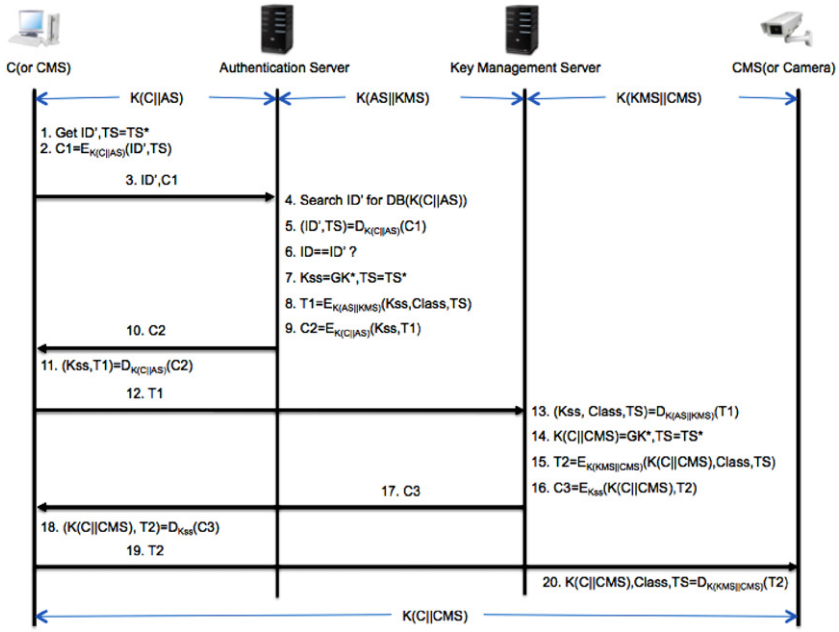

Fig. 10. Key distribution process in the surveillance video network.

Table 3
Keys in the key management system.

Key	Description
K(C‖AS)	Shared private key between client and authentication server
K(AS‖KMS)	Shared private key between authentication server and key management server
K(KMS‖CMS)	Shared private key between key management server and CMS server
Kss	Session key between client and key management server
K(C‖CMS)	Key for encrypting video image

Table 4
Key management terminology.

Term	Description
ID	Client ID
TS	Timestamp value
Class	Class code of the given ID, command codes reserved
C1	Encrypted ID based on private key between client and authentication server
C2	Encrypted session key and ticket 1
C3	Encrypted distribution key and class information and ticket 2
T1	Session ticket to KMS
T2	Service ticket to CMS
*	Related functions

Step 2. The client ID and timestamp are encrypted with the key K(C‖AS) shared between client and authentication server.
Step 3. The client ID and encrypted authentication information C1 are transmitted to the authentication server.
Step 4. The authentication server searches for the stored key in the Database using the received client ID.
Step 5. The received encrypted authentication information C1 is decrypted by the searched key K(C‖AS), which extracts the client ID and timestamp.
Step 6. The client is verified by comparing the client ID and the decrypted ID.
Step 7. Session key Kss is generated for communication between client and key management server.
Step 8. Ticket T1 is generated to connect to key management server. The generated ticket is encrypted using session key Kss. Class information for user and timestamp are encrypted using shared key K(AS‖KMS) between authentication server and key management server.
Step 9. The generated session key Kss and ticket T1 are encrypted into C2 based on shared key K(C‖AS) between client and authentication server.
Step 10. C2 is sent to the client.
Step 11. Client decrypts the received C2 using shared key K(C‖AS) between client and authentication server, and extracts session key Kss and ticket T1.

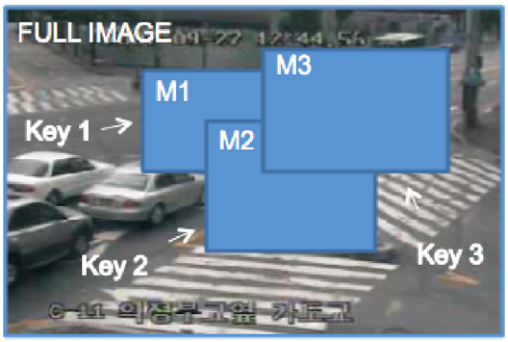

Fig. 11. Higher privileged privacy area is masked first.

Step 12. Extracted ticket T1 is transferred to key management server.
Step 13. Key management server decrypts the received ticket T1 from client using shared key K(AS‖KMS) between authentication server and key management server and extracts session key Kss and class information Class of user.
Step 14. Generate the distribution key K(C‖CMS).
Step 15. Generate ticket T2, which is generated using distribution key K(C‖CMS), class information Class of user and timestamp is encrypted using shared key K(KMS‖CMS) between key management server and video surveillance server.
Step 16. Encrypt generated distribution key K(C‖CMS) and ticket T2 using session key Kss.
Step 17. C3 is transmitted to the client.
Step 18. Client receives C3, and extracts the distribution key K(C‖CMS) and ticket T2 and decrypts C3 using session key Kss.
Step 19. Client sends ticket T2 to CMS server.
Step 20. Video surveillance server decrypts the received ticket T2 from the client and verifies distribution key K(C‖CMS), class information Class and timestamp.

A given distributed key is used in a camera or CMS server to mask images. Because higher privileged users can restore the other lower privileged privacy masks, the higher privileged privacy area is encrypted first as shown in Fig. 11. To achieve this goal, a hash-based key chain approach is used. If a user has the proper key, he can generate all of the lower privileged keys sequentially and he will be able to view the masked images. Otherwise, he will be unable to view these masked areas. When privacy masks $M_1, M_2, M_3, \ldots, M_N$ are applied to an image, it is possible for a given user with a key of a privilege level the same as M_i to restore masks M_i to M_N sequentially, but for masks M_{i-1} to M_1, it is impossible for the user to unmask them.

It is possible to implement the same mechanism in the development of an advanced networked camera to have the facility to issue a key and encrypt its images. This approach is easier because the masks can be applied to raw images.

5. Implementation results

This paper proposed a secure framework for a heterogeneous video network for surveillance management and implemented a prototype server and the integrated client module based on the introduced architecture. During the research, the current situation of the video surveillance based on the networked camera was deeply surveyed and analyzed. Through the result, a detailed architecture was derived to adopt existing heterogeneous video formats and connection protocols of the video equipment devices. Additionally, the privacy problem was resolved through a privacy mask restoration concept. According to this concept, privacy information is effectively masked along with a given privilege level and it is restored when viewed by privileged users. It would be useful when a criminal issue is involved with the masked images. Fig. 12 shows the interface of the implementation of an integrated client even though the interface may vary.

After implementation, we made a performance analysis to show the number of clients that could be supported simultaneously using our proposed integrated video surveillance system architecture. Although the accumulated delay from the total number of cascaded servers when multiple servers are deployed in real environment would be worrying, it depended too much on the network condition. Instead, we are specifically interested in measuring the server delay shown in Fig. 13 because it is the most effective factor following the revised architecture. Additionally, we estimated the performance of the video surveillance server and the web-based video surveillance clients. In order to calculate number of simultaneously supported clients by the system, we assume the following.

- We have a single video surveillance server and N clients.
- Client i accesses surveillance video i.
- Surveillance video i has size F_i bytes.
- Internet connection bandwidth of the server is B_s.
- Internet connection speed of client i is d_i.

Fig. 12. A sample implementation for the integrated client.

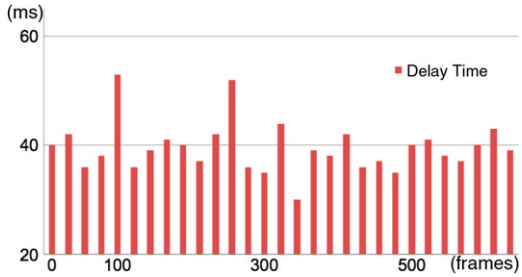

Fig. 13. Server delay.

Table 5
Comparison of system resource consumption.

	Individual client	Integrated client
CPU usage	Increased 10%	Increased 10%
Memory usage	Increased 87 M	Increased 65 M

Then we have the following. Total bytes transmitted by the surveillance video $= \sum_{i=1 \text{ to } N} F_i$ bytes, time taken by the server $T_s = \left(\sum_{i=1 \text{ to } N} F_i\right)/B_s$ time, slowest client receives at rate $d_{\min} = \min_i\{d_i\}$, largest surveillance video $F_{\max} = \max_i(F_i)$, then it takes the slowest client at least $F_{\max}/d_{\min}$ time and total transmission time: $\max\{T_s, F_{\max}/d_{\min}\}$.

In order to speed up the video surveillance transmission we need to increase the upload rate from the server, provide a higher link bandwidth at one server, deploy multiple servers, each with their own link, and deploy more infrastructures.

Table 5 shows comparison results of the rate of system resources between individual client and implemented integrated client with the implemented server. The CPU time for decoding in two procedures looks identical, with no big differences. However, there is a big difference in the memory usage. In particular, it would be a meaningful result because multiple clients need the correct same multiple memory resources for the individual client in contrast to the integrated client that needs a smaller increase along with the increase in the number of clients.

6. Conclusions

In this paper we briefly introduced the current problems and possible solutions of the video surveillance system. Due to the networked camera's lack of multiple connections and interoperability, it has consumed time and money in order to be integrated into a global large scaled surveillance video network. There should be a standard supporting interoperability as well as individual enhancement in parallel. To meet the objectives, a reasonable architectural framework was introduced and implemented in this paper. According to the comparison result, the framework showed reasonable performance and a reduced consumption of resources as well as resolving the existing problems.

References

[1] Alice Osborn, How effective are CCTV security systems at reducing crime?, Ezine Articles, 2006.
[2] Fredrik Nilsson, Intelligent Network Video: Understanding Modern Video Surveillance Systems, CRC Press, 2008.
[3] Anders Kofod-Petersen, Jörg Cassens, Proxies for privacy in ambient systems, Journal of Wireless Mobile Networks, Ubiquitous Computing, and Dependable Applications 1 (4) (2010) 62–74.
[4] K. Yim, Integrated network camera distribution server supporting heterogeneous videos and multiple users, Project report, small and medium business administration, 2007.
[5] K. Yim, Security issues on the networked video surveillance system, Technical report, Korea Internet Security Agency, 2009.
[6] AXIS media control user's manual, Axis communications, 2007.
[7] IKebe, Oqawa, Hatayanma, Network camera system using new home network architecture with flexible scalability, ICCE2005 (2005) 151–152.
[8] Kyungroul Lee, et al., Safe authentication protocol for secure USB memories, Journal of Wireless Mobile Networks, Ubiquitous Computing, and Dependable Applications 1 (1) (2010) 46–55.
[9] Brent Seals, Mathew Cutts, Active camera control from compressed image streams, Proceedings of Sensor Fusion and Networked Robotics 8 (1995) 50–57.
[10] Didier Le Gall, MPEG: A video compression standard for multimedia applicaitons, Communications of the ACM (1991) 46–58.